

Nama : Doni Nasution
Kelas : 06TPLE002
NIM : 231011400185
Mata Kuliah : Teknik Kompilasi
Perihal : UAS
Tanggal : 01-07-2026

Judul:

Simulasi Tahapan Kompilasi pada Konstruksi If-Else Menggunakan Python

1. Pemilihan Konstruksi

Konstruksi yang dipilih adalah

Percabangan (If-Else)

Contoh source code yang akan dikompilasi

```
if (x > 5) {  
    y = 1;  
} else {  
    y = 0;  
}
```

2. Pattern (Grammar)

Menggunakan BNF sederhana

$\langle \text{if_stmt} \rangle ::= \text{"if" "(" } \langle \text{condition} \rangle \text{ ")" "{" } \langle \text{statement} \rangle \text{ "}" "else" "{" } \langle \text{statement} \rangle \text{ "}"}$

$\langle \text{condition} \rangle ::= \langle \text{identifier} \rangle \langle \text{operator} \rangle \langle \text{number} \rangle$

$\langle \text{statement} \rangle ::= \langle \text{identifier} \rangle \text{"=" } \langle \text{number} \rangle$

$\langle \text{identifier} \rangle ::= \text{letter}(\text{letter}|\text{digit})^*$

$\langle \text{operator} \rangle ::= ">"$

| "<"
| "=="
| ">="

| "<="

| "!="

$\langle \text{number} \rangle ::= \text{digit}^+$

3. Program Python

```
import re

class SimpleCompiler:

    def __init__(self, source):
        self.source = source
        self.tokens = []

        # Simulasi Symbol Table
        self.symbol_table = {
            "x": "int",
            "y": "int"
        }

        # =====
        # ANALISIS LEKSIKAL
        # =====
        def lexical_analysis(self):

            pattern = r'if|else|>|=|<|=|!=|>|<|=|\\{|\\}|\\(|\\)|;|[A-Za-z_]\w*|\d+'

            self.tokens = re.findall(pattern, self.source)

            return self.tokens

        # =====
        # ANALISIS SINTAKSIS
        # Membentuk AST sederhana
        # =====
        def syntax_analysis(self):

            t = self.tokens

            # kondisi
            condition = {
                "left": t[2],
                "operator": t[3],
                "right": t[4]
            }

            # assignment THEN
            then_assign = {
                "var": t[7],
                "value": t[9]
            }

            # assignment ELSE
            else_assign = {
```

```

        "var": t[14],
        "value": t[16]
    }

    ast = {
        "type": "IF",
        "condition": condition,
        "then": then_assign,
        "else": else_assign
    }

    return ast

# =====
# ANALISIS SEMANTIK
# =====
def semantic_analysis(self, ast):

    cond = ast["condition"]

    # cek variabel kondisi
    if cond["left"] not in self.symbol_table:
        raise Exception(
            f"Error Semantik: Variabel '{cond['left']}' belum dideklarasikan."
        )

    # cek variabel assignment THEN
    if ast["then"]["var"] not in self.symbol_table:
        raise Exception(
            f"Error Semantik: Variabel '{ast['then']['var']}' belum dideklarasikan."
        )

    # cek variabel assignment ELSE
    if ast["else"]["var"] not in self.symbol_table:
        raise Exception(
            f"Error Semantik: Variabel '{ast['else']['var']}' belum dideklarasikan."
        )

    return "Analisis semantik berhasil."

# =====
# GENERASI THREE ADDRESS CODE
# =====
def generate_TAC(self, ast):

    cond = ast["condition"]

    tac = []

    tac.append(

```

```

        f'ifFalse {cond['left']} {cond['operator']} {cond['right']} goto L1"
    )

    tac.append(
        f'{ast['then']['var']} = {ast['then']['value']}'"
    )

    tac.append("goto L2")

    tac.append("L1:")

    tac.append(
        f'{ast['else']['var']} = {ast['else']['value']}'"
    )

    tac.append("L2:")

    return tac

```

```

# =====
# PROGRAM UTAMA
# =====

```

```

source = """
if ( x > 5 ) {
    y = 1 ;
}
else {
    y = 0 ;
}
"""

```

```

compiler = SimpleCompiler(source)

```

```

# 1. Analisis Leksikal
tokens = compiler.lexical_analysis()

```

```

print("===== ANALISIS LEKSIKAL =====")
print(tokens)

```

```

# 2. Analisis Sintaksis
ast = compiler.syntax_analysis()

```

```

print("\n===== ABSTRACT SYNTAX TREE =====")
print(ast)

```

```

# 3. Analisis Semantik
print("\n===== ANALISIS SEMANTIK =====")
print(compiler.semantic_analysis(ast))

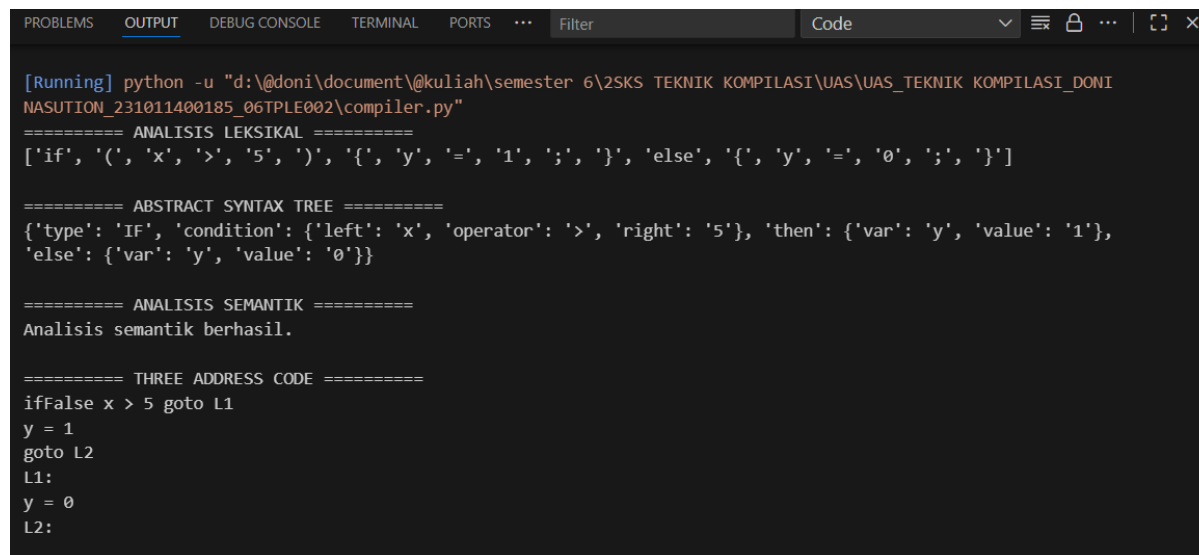
```

4. Three Address Code

```
print("\n===== THREE ADDRESS CODE =====")
```

```
for code in compiler.generate_TAC(ast):  
    print(code)
```

4. Output Program



```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS ... Filter Code  
[Running] python -u "d:\@doni\document\@kuliah\semester 6\2SKS TEKNIK KOMPILASI\UAS\UAS_TEKNIK KOMPILASI_DONI  
NASUTION_231011400185_06TPLE002\compiler.py"  
===== ANALISIS LEKSIKAL =====  
['if', '(', 'x', '>', '5', ')', '{', 'y', '=', '1', ';', '}', 'else', '{', 'y', '=', '0', ';', '}', '']  
  
===== ABSTRACT SYNTAX TREE =====  
{'type': 'IF', 'condition': {'left': 'x', 'operator': '>', 'right': '5'}, 'then': {'var': 'y', 'value': '1'},  
'else': {'var': 'y', 'value': '0'}}  
  
===== ANALISIS SEMANTIK =====  
Analisis semantik berhasil.  
  
===== THREE ADDRESS CODE =====  
ifFalse x > 5 goto L1  
y = 1  
goto L2  
L1:  
y = 0  
L2:
```

5. Penjelasan Tiap Tahapan

a. Analisis Leksikal

Tahap pertama adalah memecah source code menjadi token-token.

Contoh :	Menjadi
if (x > 5)	if (x > 5)

Token yang dikenali:

- keyword
- identifier
- operator
- angka
- symbol

b. Analisis Sintaksis

Parser memeriksa apakah urutan token sesuai grammar.

Contoh :

```
    if (x > 5){  
        ...  
    }else{  
        ...  
    }
```

akan diubah menjadi AST.

AST hanya menyimpan struktur penting tanpa simbol yang tidak diperlukan.

c. Analisis Semantik

Tahap ini memastikan program memiliki makna yang benar.

Pengecekan yang dilakukan:

- apakah variabel sudah dideklarasikan
- apakah assignment valid
- apakah kondisi dapat dibandingkan

Jika variabel belum ada, compiler akan menghasilkan error.

Contoh	akan menghasilkan
if(z > 5)	Error: Variabel z belum dideklarasikan

d. Generasi Three Address Code

AST kemudian diubah menjadi TAC.

TAC menggunakan label dan instruksi sederhana.

Contoh

```
ifFalse x > 5 goto L1
```

```
y = 1
```

```
goto L2
```

```
L1:  
y = 0
```

```
L2:
```

Tahap ini nantinya menjadi dasar sebelum menghasilkan kode mesin.

6. Kesimpulan

Program ini berhasil mensimulasikan empat tahapan utama proses kompilasi, yaitu:

1. Analisis Leksikal untuk memecah source code menjadi token.
2. Analisis Sintaksis untuk membangun Abstract Syntax Tree (AST).
3. Analisis Semantik untuk memvalidasi penggunaan variabel.

4. Generasi Three-Address Code (TAC) sebagai representasi kode antara sebelum diterjemahkan menjadi kode mesin.

Implementasi ini menunjukkan bagaimana sebuah konstruksi **if-else** diproses secara bertahap oleh compiler, sehingga mempermudah pemahaman konsep dasar teknik kompilasi.

7. Kartu Ujian



UNIVERSITAS PAMULANG

**KARTU UJIAN AKHIR SEMESTER GENAP 2025/2026 NOMOR
UJIAN : 01242836578731**

FAKULTAS / PRODI : ILMU KOMPUTER / TEKNIK INFORMATIKA S1

NAMA MAHASISWA: DONI NASUTION

NIM : 231011400185

SHIFT : REGULER C

No	Hari/ Tanggal	Waktu	Ruang	Kelas	Mata Kuliah	Paraf
1	Sabtu, 4 Jul 2026	07.40 - 09.20	V.628	06TPLE002	BASIS DATA II	1
2	Sabtu, 4 Jul 2026	07.40 - 09.20	V.628	06TPLE002	SISTEM PENDUKUNG KEPUTUSAN	2
3	Sabtu, 4 Jul 2026	09.20 - 11.00	V.628	06TPLE002	REKAYASA PERANGKAT LUNAK	3
4	Sabtu, 4 Jul 2026	09.20 - 11.00	V.628	06TPLE002	KERJA PRAKTEK	4
5	Sabtu, 4 Jul 2026	11.00 - 13.50	V.628	06TPLE002	PEMROGRAMAN II	5
6	Sabtu, 4 Jul 2026	11.00 - 13.50	V.628	06TPLE002	MOBILE PROGRAMMING	6
7	Sabtu, 4 Jul 2026	13.50 - 15.30	V.628	06TPLE002	TEKNIK KOMPILASI	7
8	Sabtu, 4 Jul 2026	16.00 - 17.40	V.628	06TPLE002	TEKNOLOGI INTERNET OF THINGS	8

Peraturan dan Tata Tertib Peserta Ujian

1. Peserta ujian harus berpakaian rapi, sopan dan memakai jaket Almamater
2. Peserta ujian sudah berada di ruangan sepuluh menit sebelum ujian dimulai
3. Peserta ujian yang terlambat diperkenankan mengikuti ujian setelah mendapat ijin, tanpa perpanjangan waktu
4. Peserta ujian hanya diperkenankan membawa alat-alat yang ditentukan oleh panitia ujian

5. Peserta ujian dilarang membantu teman, mencontoh dari teman dan tindakan-tindakan lainnya yang mengganggu peserta ujian lain
6. Peserta ujian yang melanggar tata tertib ujian dikenakan sanksi akademik



NIDN. 0418028702



Tangerang Selatan, 1 Juli 2026

Ketua Panitia Ujian

Dr. Ubaid Al Faruq, S.Pd., M.Pd.